

Benchmark AI nástrojů pro vývoj průmyslové aplikace

2D Paletizace (Single-Bin, Python 3.10+, PyQt5)

Roman Paráček, JIC Smart Factory

1. března 2026

1 Účel benchmarku

Cílem benchmarku je objektivně porovnat vybrané AI nástroje při vývoji realistické průmyslové aplikace.

Každý člen týmu implementuje aplikaci řešící problém **2D paletizace (2D bin-packing)** na jedné paletě s cílem efektivně využít plochu palety. Aplikace bude obsahovat algoritmické jádro a grafické rozhraní v prostředí PyQt5.

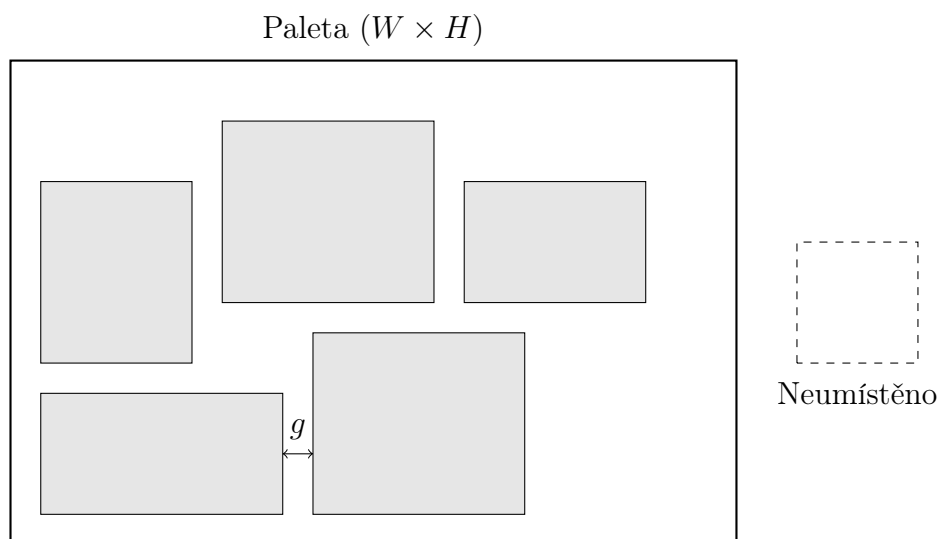
Benchmark proběhne na následujících nástrojích:

- ChatGPT – Codex
- Cursor
- Claude Code

Výsledkem bude funkční aplikace, zdokumentovaný vývojový proces a strukturované hodnocení AI nástrojů.

1.1 Ilustrace problému 2D paletizace

Obrázek 1 schematicky znázorňuje princip 2D paletizace (single-bin). Obdélník reprezentuje paletu o rozměrech $W \times H$. Menší obdélníky představují jednotlivé položky. Položky nesmí být v překryvu a musí respektovat minimální vzdálenost g .



Obrázek 1: Schematická ilustrace problému 2D paletizace (single-bin).

2 Funkční požadavky

2.1 Účel a rozsah řešení

Úkolem je implementovat řešení 2D paletizace na jedné paletě (single-bin).

Cílem je:

- zabránit překryvům položek,
- dodržet minimální mezeru mezi položkami,
- zajistit umístění položek uvnitř palety,
- maximalizovat využití plochy palety.

Řešení může být heuristické. Není požadováno nalezení globálního optima.

2.2 Vstupní parametry

Paleta (Bin)

- W – šířka palety [mm]
- H – výška palety [mm]
- g – minimální mezera mezi dvěma položkami [mm]

Omezení:

1. Každá položka musí být umístěna uvnitř oblasti $W \times H$.
2. Minimální vzdálenost mezi libovolnými dvěma položkami musí být alespoň g .
3. Mezera mezi položkou a hranou palety není vyžadována.

Položky (Items)

Pro typy položek $i = 1, \dots, n$:

- w_i – šířka footprintu [mm]
- h_i – výška footprintu [mm]
- q_i – počet kusů daného typu (libovolné nezáporné celé číslo)
- $rotation_i$ – povolení rotace o 90°

Je možné definovat libovolný počet kusů stejného typu položky. Aplikace musí umožňovat také scénář s jediným typem položky ($n = 1$), což odpovídá strukturované paletizaci.

Pokud je rotace povolena, lze použít orientaci (w_i, h_i) nebo (h_i, w_i) .

Validace vstupů

Aplikace musí:

- odmítnout záporné nebo nulové rozměry položek,
- odmítnout nekonzistentní vstupy (např. položka větší než paleta v obou orientacích),
- informovat uživatele o chybě prostřednictvím GUI a systémového loggeru,
- zabránit spuštění výpočtu při nevalidních datech.

2.3 Cíl optimalizace

Optimalizace je definována lexikograficky:

1. maximalizovat počet umístěných kusů,
2. mezi řešeními se stejným počtem kusů minimalizovat nevyužitou plochu.

Nevyužitá plocha je definována jako:

$$unused_area = W \cdot H - \sum_i (n_i^{placed} \cdot w_i \cdot h_i) \quad (1)$$

Pokud nelze umístit všechny položky, aplikace musí vrátit nejlepší validní řešení a reportovat neumístěné položky (typ a množství).

2.4 Požadovaný výstup

Aplikace musí poskytovat:

- souřadnice každého umístěného kusu,
- jeho orientaci,
- počet umístěných kusů,
- počet neumístěných kusů,
- výpočetní čas řešení,
- procentuální využití plochy.

3 Technické požadavky

- Jazyk: Python 3.10+
- GUI framework: PyQt5
- Modulární architektura (oddělení GUI a logiky)
- Typové anotace a dokumentační řetězce
- Veřejný GitHub repozitář

3.1 Požadavky na GUI a ovládání aplikace

Grafické uživatelské rozhraní (GUI) musí být navrženo přehledně, stabilně a s jasným oddělením vstupní části, vizualizační oblasti a diagnostického výstupu.

1. Zadávání vstupních parametrů

Aplikace musí umožňovat zadání všech vstupních parametrů (paleta i položky) přímo prostřednictvím GUI.

GUI musí obsahovat:

- vstupní pole pro rozměry palety (W , H),
- vstupní pole pro minimální mezeru g ,
- možnost definovat více typů položek (w_i , h_i , q_i , $rotation_i$),
- možnost přidávání a odebírání typů položek,
- podporu scénáře s jediným typem položky (strukturovaná paletizace).

Alternativně musí aplikace umožňovat načtení vstupních dat z konfiguračního souboru (např. ve formátu JSON). Formát konfiguračního souboru musí být jednoznačně popsán v dokumentaci (README).

Při zadávání parametrů musí být prováděna validace vstupů. V případě nevalidních dat nesmí být umožněno spuštění výpočtu a uživatel musí být informován prostřednictvím GUI a systémového loggeru.

2. Ovládací prvky

GUI musí obsahovat následující základní ovládací prvky:

- **Run / Start** – spustí výpočet paletizace pro aktuální vstupy,
- **Reset Scene** – vymaže aktuální vizualizaci a vrátí aplikaci do výchozího stavu,

- **Clear Logger** – vymaže obsah systémového logu.

Ovládací prvky musí být aktivní pouze v odpovídajícím stavu aplikace (např. nelze spustit výpočet při nevalidních datech).

3. Vizualizační oblast

GUI musí obsahovat grafickou oblast, ve které je zobrazena:

- paleta ($W \times H$),
- jednotlivé umístěné položky,
- jejich orientace,
- případné vizuální odlišení neumístěných položek.

Vizualizace musí být čitelná, měřítkově konzistentní a musí reflektovat skutečné výstupy algoritmu.

4. System Logger

Aplikace musí obsahovat systémový log (samostatný panel v GUI), který zaznamenává běh aplikace a slouží k diagnostice.

Logger musí:

- zobrazovat časovou značku a úroveň zprávy (INFO / WARN / ERROR),
- zaznamenávat klíčové události (načtení dat, spuštění výpočtu, dokončení, metriky),
- zaznamenávat varování (např. částečné řešení, neumístěné položky),
- zaznamenávat chyby a výjimky včetně stručného popisu,
- podporovat reset obsahu pomocí tlačítka **Clear Logger**.

5. Ošetření chyb a stabilita

GUI musí být robustní a nesmí padat při běžných chybových stavech. Veškeré chyby musí být zachyceny a korektně reportovány uživateli prostřednictvím dialogu nebo loggeru.

Aplikace musí po chybě zůstat ve stabilním a znovu použitelném stavu.

4 Struktura projektu

Repozitář musí mít strukturu:

- App/ – vstupní bod aplikace (UI)
- src/ – aplikační logika a algoritmy
- Data/ – testovací instance a případné výstupy
- Example/ – benchmark skripty
- images/ – obrázky do README
- Tests/ – automatizované testy

Součástí repozitáře musí být také pomocné skripty pro zajištění reprodukovatelného prostředí:

- **install.sh** / **install.bat** – automatizovaný instalační skript.

Skript musí:

- vytvořit nové Conda prostředí,
- nainstalovat Python (min. verze 3.10),
- nainstalovat všechny požadované knihovny,
- umožnit plně funkční spuštění aplikace bez dalších manuálních kroků.

- **verify.py** – validační skript prostředí.

Skript musí:

- ověřit verzi Pythonu,

- zkontrolovat dostupnost všech závislostí,
- otestovat import hlavních modulů aplikace,
- potvrdit, že aplikaci lze korektně spustit.

Cílem těchto skriptů je zajistit plně reprodukovatelné prostředí a bezproblémové spuštění projektu na čistém systému.

5 Obsah README.md

Součástí každého repozitáře musí být profesionálně zpracovaný soubor `README.md` v anglickém jazyce. README představuje primární vstupní dokument projektu a musí umožnit rychlé pochopení účelu, architektury a způsobu použití aplikace bez nutnosti studia zdrojového kódu.

README musí obsahovat zejména:

- jasný a strukturovaný popis projektu a jeho účelu,
- stručný kontext problému 2D paletizace,
- vizuální ukázkou aplikace (screenshot GUI),
- systémové požadavky (verze Pythonu, závislosti),
- instalační postup (včetně využití skriptů `install.sh` / `install.bat`),
- návod ke spuštění aplikace,
- popis struktury projektu,
- přehled implementovaných algoritmů a jejich principu,
- známá omezení a případné předpoklady řešení.

README musí být psán technicky přesným a profesionálním jazykem a musí odpovídat kvalitě běžné open-source dokumentace.

6 Požadované výstupy

Každý účastník benchmarku je odpovědný za dodání následujících výstupů:

1. veřejný GitHub repozitář obsahující kompletní zdrojový kód,
2. plně funkční aplikaci splňující definované požadavky,
3. README dle stanovené specifikace,
4. stručnou odbornou reflexi použití AI nástroje (max. 1 strana A4),
5. prezentaci výsledků v rozsahu 10–15 minut.

Reflexe použití AI nástroje má shrnout zkušenosti z vývoje a identifikovat přínosy i omezení nástroje v kontextu průmyslového vývoje softwaru.

7 Metodika hodnocení

Hodnocení probíhá individuálně na základě odborného posouzení. Cílem benchmarku není porovnání účastníků mezi sebou, ale kvalifikované posouzení jednotlivých AI nástrojů.

Při hodnocení se doporučuje zaměřit zejména na:

- efektivitu spolupráce s nástrojem během vývoje,
- kvalitu a udržitelnost výsledné architektury,
- míru manuálních korekcí oproti generovanému kódu,
- schopnost nástroje pracovat s kontextem projektu,
- celkovou použitelnost nástroje pro průmyslové projekty.

8 Metodika benchmarku

Benchmark je realizován formou individuální implementace definovaného zadání pomocí různých AI nástrojů. Každý účastník zodpovídá za kompletní návrh, implementaci, dokumentaci a prezentaci řešení.

Cílem benchmarku není soutěž mezi účastníky, ale získání strukturované zkušenosti s praktickým využitím jednotlivých AI nástrojů.

9 Cíl benchmarku

Cílem benchmarku je získat kvalifikovaný a strukturovaný přehled o schopnostech současných AI nástrojů při vývoji průmyslového softwaru.

Benchmark má poskytnout odpovědi na následující otázky:

- Do jaké míry jsou AI nástroje schopny samostatně navrhnout a implementovat modulární aplikaci střední složitosti?
- Jak kvalitní je generovaný kód z hlediska architektury, čitelnosti a rozšiřitelnosti?
- Jaká je reálná míra manuální korekce a zásahů?
- Jak dobře nástroje pracují s kontextem napříč více soubory?

Výsledkem benchmarku má být kvalifikované doporučení pro případné budoucí nasazení AI nástrojů v rámci vývoje průmyslových projektů.